You can do better! Using the GIN index and gin_trgm_ops class you've already created covering first_name, it's possible to search and filter using the similarity information you just learned about and find matches even for typos.

Try running the following statement in psql. Note that you're now using the SIMILARITY() function as your filter criteria.

sql/query_filter_similarity.sql

```
SELECT DISTINCT
  first_name
FROM
  users
WHERE
  similarity(first_name, 'henrieta') > 0.4;

--  first_name
-- ------------
--  Henrietta
--  Henriette
```

Although this query is much slower, it does find names that are similar. This can be improved further using Full Text Search in Rails with PostgreSQL, documented in Rails Guides.[11] By storing a list of all possible words, matches can be made using different operators to match queries against word content that is similar.

That covers it for GIN indexes. Next you'll continue with B-Tree Indexes but see how to use a Partial Index.

Using Partial Indexes

Partial indexes are B-tree indexes that include an expression in the index definition that limits the rows covered. Reducing the rows in the index means the index will be smaller which consumes less space, but also reduces index maintenance overhead at write time. In other words, as data is being written in, if it doesn't fit into the expression for a partial index, it won't be included in the index.

A partial index is a specialized index because it's optimized for certain values within a column, at the expense of not being useful for others.

Here's an example to help clarify this. Imagine that the Rideshare application uses "soft deletes" when rows are deleted in the app. As a refresher, "Soft Deletes" means rows aren't actually deleted but are instead updated to make them hidden from the application.

11. https://guides.rubyonrails.org/active_record_postgresql.html#full-text-search